

PostQode Context Management — Architecture

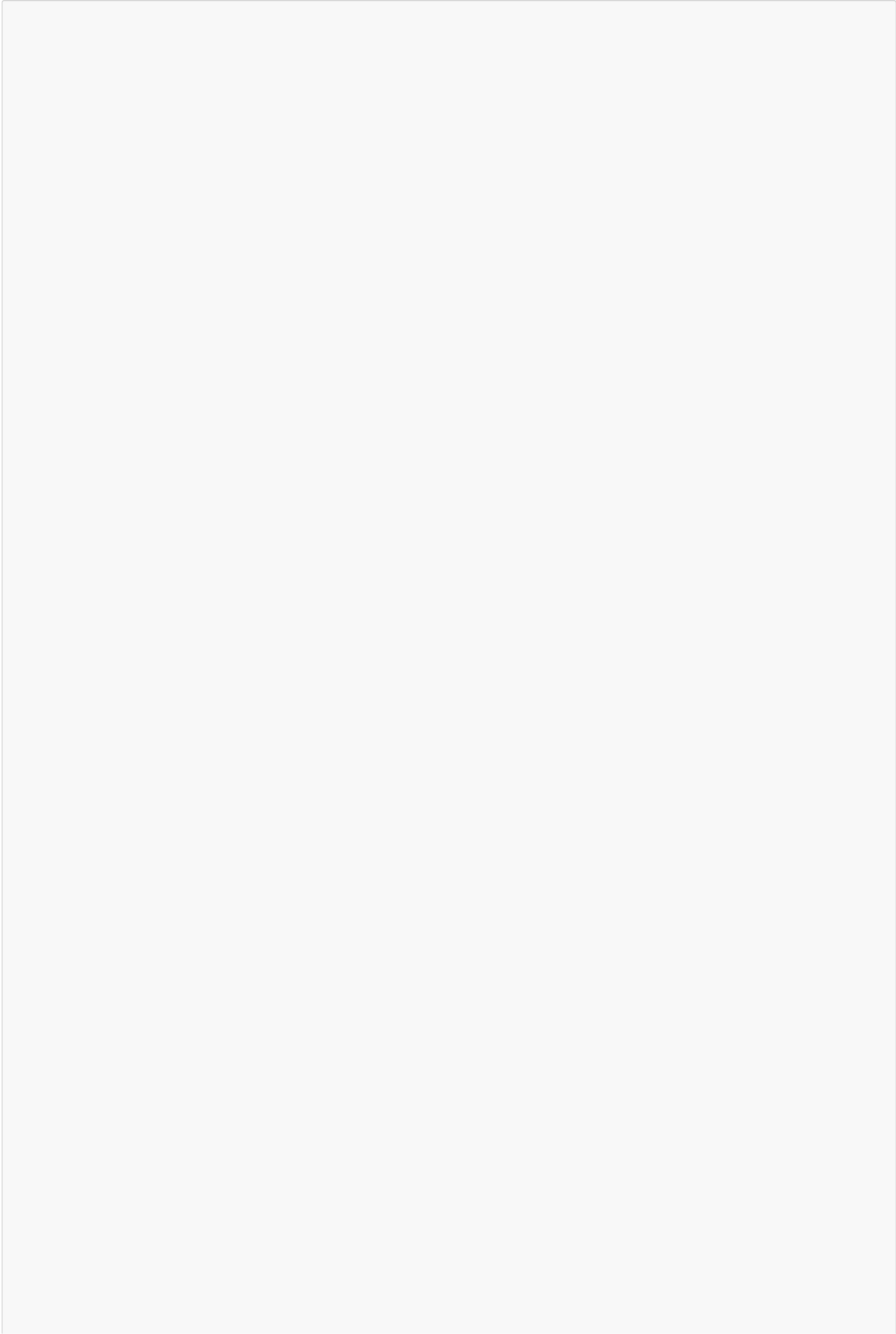
Last updated: 2026-08-05 · Supersedes [CONTEXT_MANAGEMENT_REVIEW.md](#) (the pre-fix review notes). Scope: [src/core/context/context-management/*](#), capture-time caps across tool handlers and terminal managers, system-prompt pinning in [src/core/task/index.ts](#), provider cache routing in [packages/postqode-ai](#), and the SDK loop in [packages/postqode-agent](#).

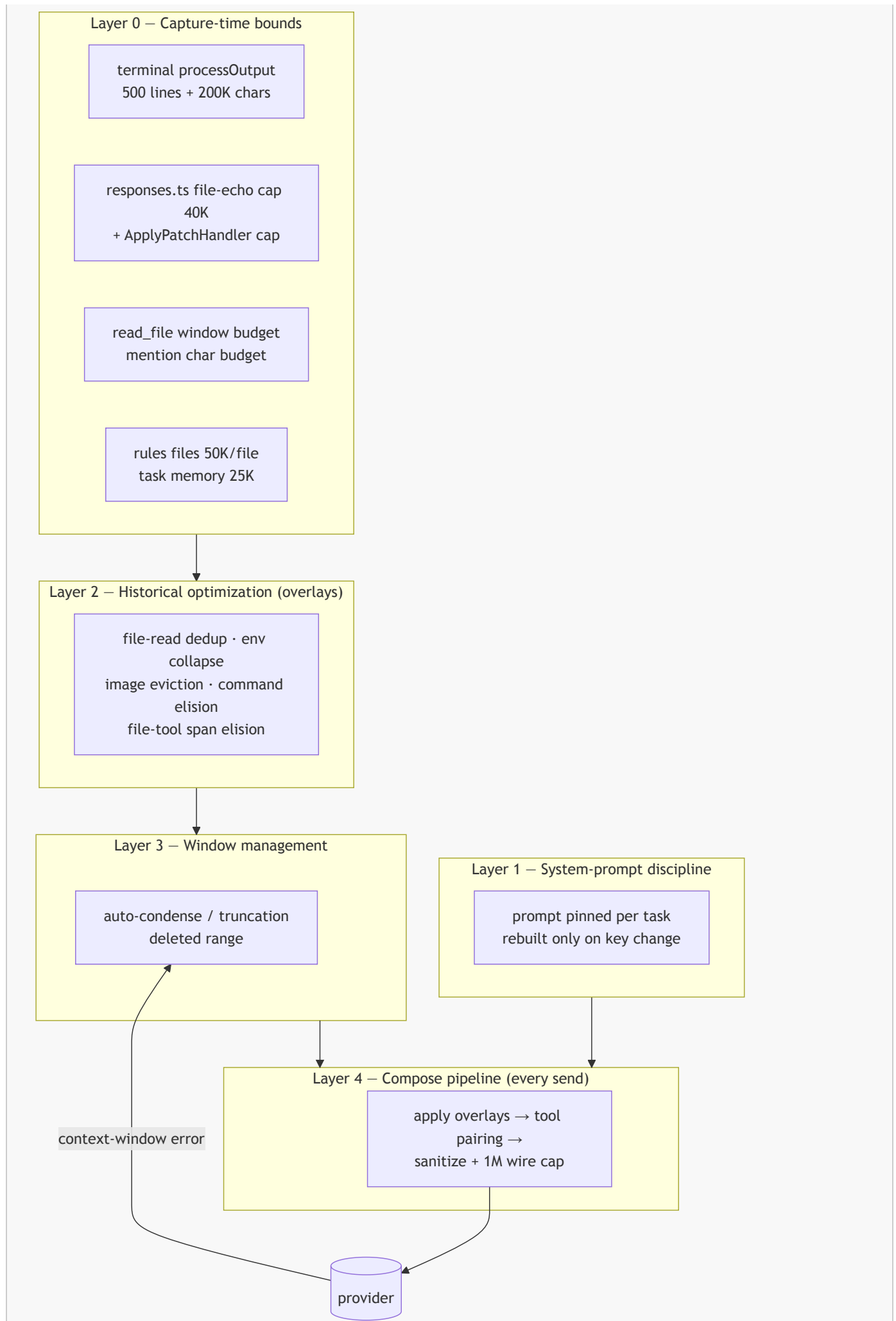
1. Problem statement and design goals

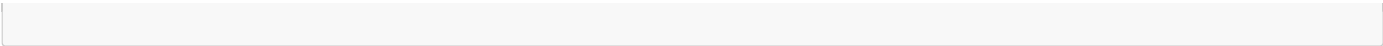
PostQode keeps a full, append-only API conversation history per task. Every request must:

1. **Fit the model's context window** — and degrade gracefully (collapse → condense → truncate) rather than fail when it doesn't.
2. **Fit provider transport limits** — which are *per-item* and far below the window (OpenAI rejects any single text item over 10 MiB, regardless of the window). A block that big is persisted history by the time it is sent, so without a guard every auto-retry replays the same rejection and the task wedges permanently.
3. **Not thrash the prompt cache** — prompt caches are prefix-keyed; rewriting the bytes of ANY older message re-bills the whole prefix at the cache-WRITE rate (1.25×) instead of the cache-READ rate (0.1×). Measured on the PostQode gateway, per-token price more than doubles between a 58K and a 446K prompt while cache effectiveness falls from 5.3× to 1.85×.
4. **Survive resume, retry, and checkpoint restore** — all state is either derived from the append-only history or persisted with validation and atomic writes.

The architecture is **defense in depth**: five layers, each bounded and fail-safe, so no single content source or lifecycle path can produce an unbounded or unrecoverable request.







2. Core data model

The raw `apiConversationHistory` is **never mutated**. Two overlay layers are composed at send time (`ContextManager.getAndAlterTruncatedMessages`):

Layer	State	Semantics
Deleted range	<code>conversationHistoryDeletedRange: [start, end]</code> (task metadata)	One contiguous window masked out at send time. Always keeps the first user/assistant pair (indices 0–1); removes an even number of messages from index 2; snaps the end to an assistant boundary so user/assistant alternation survives. Truncation <i>masks</i> ; it never splices, so message indices are stable for the task's lifetime.
Overlay updates	<code>contextHistoryUpdates: Map<msgIndex, [EditType, Map<blockIndex, ContextUpdate[]>]></code>	Per-block rewrites applied at compose. <code>ContextUpdate = [timestamp, updateType, MessageContent, MessageMetadata]</code> . Timestamps support checkpoint rollback (<code>truncateContextHistoryAtTimestamp</code>).

`EditType` records which pass first claimed a message: `NO_FILE_READ`, `READ_FILE_TOOL`, `ALTER_FILE_TOOL`, `FILE_MENTION`, `ENVIRONMENT_DETAILS`, `IMAGE`, `COMMAND_OUTPUT` (7), `FILE_TOOL_CONTENT` (8).

Write discipline. Every pass funnels overlay writes through `recordBlockUpdates`, which owns the four safety behaviors: *noteUndo before* the get-or-create (so an unprofitable round is byte-exactly revertible), `EditType`-preserving get-or-create, per-block idempotency (`innerMap.has(blockIndex) → skip` — an overlay is never overwritten), and the canonical single-entry write. The mention-continuation path appends instead of replacing and copies (never aliases) the previous round's metadata array, so length-based revert stays exact.

Persistence. `context_history.json` is a versioned envelope `{version: 1, updates: [...]}` (legacy bare arrays still load; legacy 3-element updates are normalized to 4-tuples). Writes are atomic — unique tmp name (`<file>.<pid>-<n>.tmp`) + rename — with orphan sweeps on task init and task delete. Loading validates each entry down to individual `ContextUpdate` fields (including every metadata element as `string[]`); malformed entries are dropped individually and loudly, never crashing compose mid-request. This matters because the deleted range persists *separately*: a lost overlay map with a surviving deleted range would send a truncated history with no notice telling the model so.

3. The per-turn pipeline (`getNewContextMessagesAndMetadata`)

Runs on every request. Order matters:

- Cache-awareness + threshold.** `cacheAware = supportsPromptCache || /claude/i.test(modelId)` (the id check mirrors what the wire transform keys on, because catalogue entries can under-report). `effectiveThreshold = useAutoCondense ? min(contextWindow × autoCondenseThreshold, maxAllowedSize) : maxAllowedSize`.
- Pressure gate.** Optimization passes run only when the last request's usage $\geq \text{OPTIMIZATION_PRESSURE_RATIO} (0.8) \times \text{effectiveThreshold}$. Below that, the history stays byte-identical turn over turn and the cached prefix keeps hitting. Non-cache-aware providers pay nothing for rewrites and keep the old always-optimize behavior.
- Unprofitable-round memo.** A round over unchanged inputs is deterministic. After a round that found nothing or was reverted, its state key (`cacheAware:historyLength:deletedEnd:overlayEntryCount`) is remembered and the round is skipped until the key changes — unless truncation is imminent (a truncating turn keeps even an unprofitable round, because the prefix dies anyway and collapsing shrinks the post-truncation prefix).
- Optimization round (under an undo log).** `applyContextOptimizations` runs the five passes (§4), then `calculateContextOptimizationMetrics` measures the savings ratio.
- Truncation decision** (non-condense path). Reads the **exact** record at `previousApiReqIndex` — not the walk-back — because truncation is irreversible and a zero-usage (cancelled) turn must mean "unknown, don't truncate", not "reuse an older pre-truncation total and re-cut". Truncates half (or quarter when `total/2 > maxAllowedSize`, e.g. after a model switch to a smaller window) unless the round just reclaimed $\geq \text{MIN_OPTIMIZATION_PAYOFF_RATIO} (0.3)$.
- Payoff gate.** For cache-aware models, a round that reclaimed < 0.3 of in-range characters and did not coincide with truncation is **reverted byte-exactly** (`revertOptimizationRound` restores captured array lengths/absence), so the wire bytes match the previous request and the prefix survives. The break-even math: collapsing N tokens of a P-token prefix repays its invalidation only after $T \geq (1.15P - 1.25N) / 0.1N$ turns — ~26 turns at 30% savings, ~217 at 5% — hence both the pressure gate (batch collapses near the cliff) and the payoff gate (never keep a small collapse).
- Single persist point.** Disk is written only when a truncation notice was written or a *kept* round produced updates. A reverted round restored memory to exactly what is on disk.
- Compose** (§6) and return.

One diagnostic line per turn: `[ContextOpt] usage=... pressure=... cacheAware=... skipped=... updates=... saved=... truncated=... reverted=...` (visible in the "PostQode" output channel, which `Logger.subscribe` now feeds).

4. The optimization passes

All passes share the contract: non-destructive (overlays only), idempotent (claimed blocks are skipped), undo-logged, and **never write inside the trailing safety window** — `PROTECTED_RECENT_MESSAGES` (4) messages, counted in *messages* not blocks because parallel tool calling (up to 10 concurrent) can put many results in one turn, and because overlays are keyed by absolute index while resume/checkpoint flows drop trailing messages (an overlay at a vacated index would be inherited by different content).

Pass	Recognizes	Collapses to	Retention
File-read dedup	<code>[read_file for 'x']</code> Result: blocks, <code>alter</code> results, <code><file_content path=...></code> mentions (cached structural scan per message, identity-revalidated)	Duplicate-read notice	The latest read of each file always survives; mention continuation collapses one more file per round via metadata bookkeeping
Env-details collapse	<code><environment_details></code> blocks (one per user turn)	Stale-snapshot notice	Latest snapshot intact
Image eviction	Tool-produced images only (<code>EVICTABLE_IMAGE_TOOLS</code> allowlist; attribution via <code>tool_use_id</code> or XML header; unattributable = user-attached = never evicted)	Stale-image notice	Newest <code>KEEP_LATEST_IMAGE_BLOCKS</code> (3)
Command-output elision	<code>[execute_command for '...']</code> Result: + a byte-exact status line. Anchoring is spoof-safe: the first user-feedback marker bounds every search, and the header close is the last <code>']</code> Result: before that boundary — a command or output that spoofs the format can only shrink the window (under-elision), never slice user bytes	Head 600 + notice + tail 1200 chars of the body (head keeps setup errors, tail keeps verdicts)	Newest <code>KEEP_LATEST_COMMAND_RESULTS</code> (3); bodies < <code>COMMAND_OUTPUT_ELISION_FLOOR</code> (6000) chars left alone
File-tool span elision	<code>[replace_in_file/write_to_file for '...']</code> and <code>[apply_patch for ...]</code> results — current single-block layout <i>and</i> the legacy header-only-block-0/content-block-1 layout	The span from the FIRST <code><final_file_content>/<file_content></code> open tag to the LAST close tag becomes one notice. Span-based because echoed file bodies can contain literal close tags (this repo's own sources do), which defeat any per-element regex. Within a file-tool block everything in that span is tool-emitted; user feedback rides in a separate block that is never scanned	Spans < <code>FILE_TOOL_CONTENT_ELISION_FLOOR</code> (= <code>MAX_FINAL_FILE_CONTENT_CHARS</code> , 40K) stay

The file-tool pass exists for **historical** echoes: `responses.ts` and `ApplyPatchHandler` stopped emitting full bodies above 40K (\$5), but histories written before that cutoff — the observed case: three ~140K `replace_in_file` echoes ≈ 420K persistent chars dominating a 633K-token task — still need collapsing.

5. Layer 0 — capture-time bounds (the case matrix)

Every content source that enters the conversation, and its bound:

Source	Capture bound	Historical collapse	Notes
<code>execute_command</code>	500 lines and <code>MAX_TERMINAL_OUTPUT_CHARS</code> (200K) in both <code>processOutput</code> impls (VS Code + standalone)	command elision	Char cap added because line limits pass multi-MB single lines (grep over a minified bundle → the observed 17.4MB result)
Env-details terminal output (<code>getUnretrievedOutput</code>)	<code>capOutputChars</code> at both injection sites	env collapse	Raw process output, no line limit upstream
<code>replace_in_file</code> / <code>write_to_file</code>	> 40K body omitted with re-read instruction (<code>formatFinalFileContent</code>	file-tool span elision	<code>MAX_FINAL_FILE_CONTENT_CHARS</code> exported and shared with the elision floor

Source	Capture bound	Historical collapse	Notes
	/ <code>formatRevertedFileContent</code>); byte-identical writes omitted entirely		
<code>apply_patch</code>	Same 40K cap per file in <code>ApplyPatchHandler</code>	file-tool span elision	Was the one uncapped echo path
<code>read_file</code>	Remaining-window budget (<code>computeContentCharBudget</code> , 100K fallback)	file-read dedup	
@-mentions	Same char budget	mention continuation	
Workspace file listing	200 entries	—	
<code>search_files</code> / <code>list_files</code>	ripgrep 300 results + byte caps / 200 entries	—	
Rules files → system prompt	<code>MAX_RULE_FILE_CHARS</code> (50K) per file at every read site	— (system prompt never passes the wire cap, so this is the only bound)	An oversized rules file would otherwise ride in every request of every task in the workspace
Task memory → system prompt	25K / 200 lines (<code>truncateEntrypointContent</code>)	—	
Subagent results (sync)	4K excerpt per agent	—	
SDK/coding-agent <code>bash</code> tool	200K cap in the tool itself	SDK compaction	The SDK loop has no compose pipeline; the cap lives at capture
User pastes, MCP results, browser snapshots/eval/console, <code>web_fetch</code> , <code>use_skill</code> bodies, background-task outputs	Wire cap only (by design)	—	Bounded to 1M chars per block; one occurrence costs up to ~250K tokens but can never wedge a task

6. Layer 4 — the compose pipeline and the wire guard

`getAndAlterTruncatedMessages` runs on **every** send, including the first request of a task and the auto-condense summarization request:

- 1. **Apply overlays** over the masked range (clones before mutating; originals untouched).
- 2. **ensureToolResultsFollowToolUse** — repairs `tool_use/tool_result` pairing across the truncation boundary in both directions (orphan conversion + stub insertion), so no pass or cut can produce the API-rejected orphan shapes.
- 3. **sanitizeMessagesContent** — drops malformed (null/typeless) blocks persisted by interrupted streams, and enforces `MAX_WIRE_TEXT_BLOCK_CHARS` (1M) per text block / `tool_result` content / string message, copy-on-write with head+tail retention and an actionable notice. This is the last line of defense against the permanent-wedge failure mode: an oversized *persisted* block sits inside every recency protection and survives truncation, so it must be capped at send time.

Subagent runs (`AgentRunner`) do not use `ContextManager`, but apply the same `sanitizeMessagesContent` guard directly before `createMessage`.

7. System-prompt pinning

The system prompt concatenates disk state (rules, skills catalogue, `MEMORY.md`, `.postcodeignore`) — rebuilding it per turn meant any file change mid-task silently rewrote the prompt and invalidated the entire prefix. It is now **built once per task and pinned**, rebuilt only when `buildSystemPromptKey` (`src/core/task/system-prompt-key.ts`, pure and unit-tested per dimension) changes:

- **In the key** (changes prompt bytes AND can change mid-task through the UI): mode, provider, model id, custom prompt, native tool calls, browser support, web tools, focus-chain settings, viewport, MCP server names (order-insensitive), skill toggles (global + local), yolo mode, preferred language, `.postcodeignore` content, workspace root paths.
- **Explicit invalidation** (`Task.invalidateSystemPrompt()`): rule create/delete/toggle RPCs and skill create/delete (a default-enabled skill has no toggle entry, so key perturbation alone can't see it). Workflow create/delete deliberately does **not** invalidate — workflows feed slash commands, not the prompt.
- **Deliberately excluded** (the tradeoff pinning exists to make): plain edits to rules files, `MEMORY.md`, and the on-disk skill catalogue take effect on the next task, not the next turn.

The `:1m` context-window patch runs on **every** request, outside the pin-miss branch — the catalogue object it mutates is TTL-cached and replaced by `refreshOpenRouterModels`, so a once-per-pin patch would leave later turns computing truncation thresholds against a 200K window and repeatedly cut healthy 1M-context conversations.

8. Provider cache routing (`packages/postqode-ai`)

- **Two caching mechanisms:** Anthropic-family models need explicit `cache_control` breakpoints (system message + last two user messages, ≤ 4 total); OpenAI/DeepSeek/xAI-family models cache automatically but route on `prompt_cache_key` — sent as `pg_<sha256(systemPrompt)[:32]>`, stable across turns and task resume, changing exactly when the prompt does. Gating is on `supportsPromptCache || claude-id`, not on `anthropic/` prefixes. (Open item: one live gateway request per model family to confirm non-Anthropic upstreams tolerate the `cache_control` parts.)
- **Sticky provider:** OpenRouter-style routers load-balance across upstreams, but caches are provider-local. The first response's `provider` field (validated charset, ≤ 64 chars) is latched for the task and sent as `provider.order[[sticky]]` with `allow_fallbacks: true` — a cost preference, not an availability constraint. Precedence: `:1m` pin > per-model preferences > sticky > user sort. Known tradeoffs: latch-once (no re-latch on failover) and it outranks the user's sort from turn 2.
- **Exclusive usage buckets:** consumers compute occupancy as `in + out + cacheWrites + cacheReads`, so providers whose `prompt_tokens` is inclusive of cache buckets (LiteLLM/OpenAI convention, DeepSeek hit-miss) have the cached portion subtracted from `inputTokens` — previously double-counted, firing auto-condense at roughly half the intended threshold. The clamp to 0 logs loudly when a proxy reports already-exclusive buckets.
- **Accounting:** `sumRequestTokens` / `getLatestRequestTokenUsage` are the single source of truth (walk-back past zero-usage records for the pressure/condense gates and the GUI bar; exact-record-only for irreversible truncation). Per-request `[Cache] $/Mtok` logging in `task/utils.ts` — cost-derived, because the gateway's self-reported bucket split has been observed lying.

9. Lifecycle paths and recovery

All request-sending paths and their protection:

Path	Compose + wire cap	Recovery
Normal loop (<code>recursivelyMakePostQodeRequests</code> → <code>attemptApiRequest</code>)	yes	3 metered auto-retries, then user ask
Auto-retry (first-chunk and mid-stream reinit)	yes — recomputed each attempt	bounded by <code>autoRetryAttempts ≤ 3</code>
Auto-condense summarization request	yes — a task wedged by an oversized block can condense out	2 condense failures → hard-truncate half (keeps the task message: <code>replaceFirstUserMessage=false</code>)
Manual <code>/condense</code> , <code>/snip</code>	range/overlay mutation only; next send re-enters normal loop	snip keeps the task message (no summary exists)
Context-window-exceeded error	truncation alternates with metered retries; shrinks monotonically	first-user replacement only on condense paths
Resume from disk	overlays reloaded + validated (<code>initializeContextHistory</code>), orphan tmp sweep	inherits normal-loop recovery
Checkpoint restore	history sliced, overlays loaded then timestamp-truncated (a fresh unloaded manager would wipe the map), task reinitialized	inherits resume
Subagents (<code>AgentRunner</code>)	wire guard applied directly at <code>createMessage</code>	CWE → compact + retry (≤ 3)
SDK loop (<code>@postqode/agent</code>)	no compose pipeline; bash capture cap closes the realistic vector	compaction (reserve-based) → fail-fast on API error

No path retries forever; every truncation shrinks monotonically.

10. Invariants

1. **Raw history is append-only and never mutated** — overlays and masks only; compose is copy-on-write.
2. **User-authored bytes are never elided** — by construction (feedback-boundary anchoring, separate-block scanning, protected windows), not by pattern-matching that hostile content could defeat. Spoofed formats can only cause under-elision.

3. **An overlay is written at most once per block** and never inside the trailing 4 messages.
4. **A rewrite of history must earn its cache invalidation** ($\geq 30\%$ reclaimed) or coincide with truncation — otherwise it is reverted byte-exactly.
5. **Every send path is bounded per-block** (1M wire cap) and every capture path is bounded at source; no single content source can exceed provider transport limits.
6. **Truncation never fires on unknown usage** and never removes the first user/assistant pair; the task statement survives every no-summary truncation.
7. **Corrupt persisted state degrades, never crashes**: per-entry validation drops bad data loudly; atomic writes make torn files impossible; an unreadable file yields an empty overlay map, not an exception mid-request.

11. Known tradeoffs (accepted, documented in code)

- Stale env/file duplicates inside the protected window wait ≤ 2 turns before collapsing.
- Multi-file `apply_patch` spans collapse interleaved per-file summaries along with the echoes.
- A pre-cap oversized echo whose body embeds a literal close tag *and* is later re-read can be claimed by the dedup pass first and under-elide (missed savings, never corruption).
- Sticky provider: latch-once, outranks user sort from turn 2.
- `createRuleFile` invalidates while the new file is still empty; the content edit that follows waits for the next task.
- BACKSTOP-ONLY sources (§5) can each cost up to ~250K tokens per occurrence before collapse.

12. File map

Concern	Files
Core engine	<code>src/core/context/context-management/ContextManager.ts</code>
Wire guard	<code>src/core/context/context-management/sanitize-message-content.ts</code>
Overflow-error classification	<code>src/core/context/context-management/context-error-handling.ts</code>
Per-turn wiring, pinning, :1m patch	<code>src/core/task/index.ts</code> , <code>src/core/task/system-prompt-key.ts</code>
Cache diagnostics	<code>src/core/task/utils.ts</code> ([Cache]), <code>src/activation/initializer.ts</code> (Logger → output channel)
Capture caps	<code>src/integrations/terminal/constants.ts</code> (<code>capOutputChars</code>), both <code>processOutput</code> impls, <code>src/core/prompts/responses.ts</code> , <code>src/core/task/tools/handlers/ApplyPatchHandler.ts</code> , <code>src/core/task/TaskContextSetup.ts</code> , <code>src/core/context/instructions/user-instructions/rule-helpers.ts</code>
Provider cache work	<code>packages/postqode-ai/src/transform/openrouter-stream.ts</code> , <code>providers/{openrouter,postqode,postqode-trial,litellm}.ts</code>
SDK compaction + bash cap	<code>packages/postqode-agent/src/compaction.ts</code> , <code>packages/postqode-coding-agent/src/tools/bash.ts</code>
Subagent guard	<code>src/core/agents/runtime/AgentRunner.ts</code>
Tests	<code>src/core/context/context-management/__tests__/ContextManager.test.ts</code> (78), <code>sanitize-message-content.test.ts</code> , <code>context-error-handling.test.ts</code> , <code>responses.test.ts</code> , <code>system-prompt-key.test.ts</code> , <code>rule-helpers.test.ts</code> , <code>output-cap.test.ts</code> , plus package suites — all wired into <code>vitest.sdk.config.ts</code> (<code>npm run test:sdk</code>)